

Map-STDP

Vikram Ramavarapu¹

¹Computer Science, University of Illinois, Urbana, Illinois, USA.

Contributing authors: vikramr2@illinois.edu;

Abstract

STDP is a biologically plausible learning rule that has been widely used in spiking neural networks. However, STDP is a local learning rule that cannot capture the global modular structure of the neural network. In this paper, a new Map-STDP rule is derived using the map equation, which is an info-theoretic objective function for community detection.

1 Introduction

Spike-timing dependent plasticity (STDP) [1] is a biologically plausible learning rule that has been widely used in spiking neural networks. When a pre-synaptic neuron fires shortly before a post-synaptic neuron, the synapse between them is strengthened. Conversely, if the pre-synaptic neuron fires shortly after the post-synaptic neuron, the synapse is weakened.

However, STDP is a fully local rule that cannot capture the global structure of the neural network. This issue is much more prominent when population vectors are used for inference via the population of neurons in different regions of the network. In order to ensure the expressivity of the population vectors, the neural network must have a modular structure, much like the cortical organization of the brain.

Moreover, neural dynamics are very stochastic and operate as a form of random walk [2], which is a major component of the map equation, an info-theoretic objective function for community detection in the Infomap algorithm [3]. The map equation measures the modular structure of a network by quantifying the amount of information needed to describe the flow of activity through the network via a random walk.

As such, a Map-STDP rule that incorporates the map equation is derived. The Map-STDP rule is a local learning rule that can capture the global modular structure of the neural network. Moreover, the Map-STDP rule preserves biorealism through

the use of info-theoretic entropy maximization, stochastic neural dynamics, and local synaptic updates.

2 Infomap and Entropy

2.1 Infomap

The Infomap algorithm is a community detection algorithm that minimizes the map equation to identify communities in a network. The map equation is defined as:

$$L(M) = q_{\cap} H(\mathcal{Q}) + \sum_{m \in M} p_{\cap}^m H(\mathcal{P}^m)$$

where M is a partition of the network into m communities. There are two terms. The first term represents the entropy of movement between communities, while the second term represents the entropy of movement within communities, where exiting the community is also considered a movement.

2.2 Neurological Entropy

The brain also operates with two different types of entropy. The first is response entropy of the individual neurons [4]. Maximizing this entropy ensures that the neurons are utilizing their full dynamic range, so that they are as expressive as possible. Formally, for a neuron with firing rate r , and maximum firing rate r_{max} , the response entropy is defined as:

$$H = \log_2 \left(\frac{r_{max}}{\Delta r} \right)$$

assuming a uniform distribution of firing rates, where Δr is the resolution of the firing rate. With this entropy value emerges mutual information between the stimulus and the neural response, which is a measure of how much information the neural response conveys about the stimulus. This is formalized as:

$$I(r; a) = H - \langle H_a \rangle_a$$

The second is thermodynamic entropy, otherwise known as free energy [5]. Given p , the brain's probabilistic model of the world, and q , the brain's recognition density, the free energy is defined as:

$$F = \mathbb{E}_q[-\log(z, o)] - H(q(z))$$

Where z is the latent variables and o is the sensory input. The first term is the expected negative log likelihood of the brain's model of the world, which is also known as the energy. The second term is the entropy of the brain's recognition density. This can be expanded further as:

$$H(q) = \mathbb{E}_q[-\log q(z)]$$

Therefore,

$$F = \mathbb{E}_q[\log q(z) - \log p(z, o)] = D_{KL}(q(z) || p(z, o)) - \log p(o)$$

2.3 Connecting the Map Equation to Neurological Entropy

The map equation can be connected to both neurological entropies mentioned above through the theme of surprise minimization. Response entropy minimizes surprise at the neural level, while free energy minimizes surprise at the network level. The map equation addresses the meso-scale level.

First, with response entropy, standard STDP already maximizes mutual information between the stimulus and the neural response. This means that the Map-STDP rule would be optimally utilized when compounded with an STDP rule that maximizes response entropy, while the map equation term acts as a modular gating mechanism.

Second, with free energy, the map equation can be seen as operating on the same underlying distribution that free energy minimization converges to. Specifically, the stationary distribution that emerges from the SNN's MCMC dynamics, $p(\mathbf{z}|\mathbf{o})$ [2], is precisely the distribution that minimizes variational free energy in Equation ??, that is, the optimal recognition density $q(\mathbf{z}) = p(\mathbf{z}|\mathbf{o})$ at the free energy minimum. The map equation operates on the marginal visit frequencies $\tilde{p}_i = p(z_i = 1|\mathbf{o})$ that emerge as a byproduct of this same sampling process. In this sense the map equation and free energy minimization are complementary: free energy minimization defines what the network should represent, while the map equation defines how the mesoscale structure of the network should be organized to represent it efficiently.

3 Deriving Map-STDP

As mentioned earlier, in order to properly address response entropy, the Map-STSP rule should be compounded with an STDP rule that maximizes response entropy. As such, we can start with a rule in the form of:

$$\Delta W_{ij} = \eta \cdot \text{STDP}(i, j) - \eta' \cdot G(i, j)$$

where $\text{STDP}(i, j)$ is a standard STDP rule that maximizes response entropy, $G(i, j) = \partial L / \partial W_{ij}$ is the gradient of the map equation, and η, η' are separate learning rates for the two objectives. The two terms are additive rather than multiplicative, so each objective acts independently: STDP drives the synaptic updates toward maximizing response entropy, while the map gradient term descends the map equation loss to encourage modular structure. The goal is to derive the form of $G(i, j)$.

In its general form, a random walk on a network has a transition matrix T that can be defined as:

$$T_{ij} = \frac{W_{ij}}{\sum_k W_{ik}}$$

where W_{ij} is the weight of the edge from node i to node j . We can then tie this in to Buesing et. al.'s equation for the membrane potential of a neuron in an SNN that performs MCMC sampling [2]:

$$u_i = b_i + \sum_j W_{ij} z_j(t)$$

where u_i is the membrane potential of neuron i , b_i is the bias of neuron i , and $z_j(t)$ is the binary spike variable of neuron j at time t . While z is an instantaneous variable, we can take its expectation over time in order to make it compatible with the map equation's flow-based framework. This gives us:

$$\langle z_j \rangle = p(z_j = 1 | \mathbf{o})$$

This gives us the corrected definition of our transition matrix as:

$$\tilde{T}_{ij} = \frac{W_{ij} \langle z_j \rangle}{\sum_k W_{ik} \langle z_k \rangle} = \frac{\tilde{W}_{ij}}{\tilde{s}_j}$$

With our new defined variables $\tilde{W}_{ij} = W_{ij} \langle z_j \rangle$ and $\tilde{s}_j = \sum_k W_{ik} \langle z_k \rangle$.

The Chapman-Kolmogorov equation then gives us the probability of being at node j at time t as:

$$\pi_j^{t+1} = \sum_i \tilde{T}_{ij} \pi_i^t$$

And when applied to the stationary distribution of our neural random walk, $\tilde{p}$, we get the following fixed point equation:

$$\tilde{p}_j = \sum_i \tilde{T}_{ij} \tilde{p}_i \rightarrow \tilde{p} = \tilde{T} \tilde{p}$$

We can rule out the trivial solution of $\tilde{p} = 0$ by enforcing a normalization constraint of $\sum_j \tilde{p}_j = 1$. Another trivial solution we can rule out is the one where $\tilde{T} = I$, which could only be true if we don't add a constraint of irreducibility, meaning there must be some level of connectivity between all nodes in the network. With these constraints in place, we can then solve for $\tilde{p}$ using power iteration:

$$\tilde{p}^{(k+1)} = \tilde{T} \tilde{p}^{(k)}$$

Which can be estimated online via the local update rule:

$$\tilde{p}_i \leftarrow (1 - \beta) \tilde{p}_i + \beta \sum_j \tilde{T}_{ij} z_j(t) \quad (1)$$

The proofs of convergence are deferred to [Appendix A](#).

Now, it's time to come back to the map equation and derive the form of $G(i, j)$. We can rewrite $p_{\odot}^m$ and $q_{\curvearrowright}$ in terms of $\tilde{p}$:

$$\begin{aligned} p_m &:= \sum_{i \in m} \tilde{p}_i \\ q_m &:= \sum_{i \in m} \sum_{j \notin m} \tilde{T}_{ij} \tilde{p}_i \\ p_{\odot}^m &= p_m + q_m \\ q_{\curvearrowright} &= \sum_{m \in M} q_m \end{aligned} \quad (2)$$

Where p_m is the total visit frequency of community m , and q_m is the total exit frequency of community m . We can then rewrite the map equation as:

$$L(M) = q_{\curvearrowright} H(\mathcal{Q}) + \sum_{m \in M} (p_m + q_m) H(\mathcal{P}^m)$$

Now we need to expand the entropy terms.

$$H(\mathcal{Q}) = - \sum_{m \in M} \frac{q_m}{q_{\curvearrowright}} \log \left(\frac{q_m}{q_{\curvearrowright}} \right)$$

$$H(\mathcal{P}^m) = - \frac{q_m}{p_m + q_m} \log \left(\frac{q_m}{p_m + q_m} \right) - \sum_{i \in m} \frac{\tilde{p}_i}{p_m + q_m} \log \left(\frac{\tilde{p}_i}{p_m + q_m} \right)$$

Now, substituting this back into the map equation gives us:

$$\begin{aligned} L(M) &= - \sum_{m \in M} q_m \log \left(\frac{q_m}{q_{\curvearrowright}} \right) \\ &\quad - \sum_{m \in M} (p_m + q_m) \left[- \frac{q_m}{p_m + q_m} \log \left(\frac{q_m}{p_m + q_m} \right) - \sum_{i \in m} \frac{\tilde{p}_i}{p_m + q_m} \log \left(\frac{\tilde{p}_i}{p_m + q_m} \right) \right] \\ &= - \sum_{m \in M} q_m \log \left(\frac{q_m}{q_{\curvearrowright}} \right) + \sum_{m \in M} q_m \log \left(\frac{q_m}{p_m + q_m} \right) + \sum_{m \in M} \sum_{i \in m} \tilde{p}_i \log \left(\frac{\tilde{p}_i}{p_m + q_m} \right) \end{aligned}$$

Now, we can take the derivative of $L(M)$ with respect to W_{ij} to get the form of $G(i, j)$. Since $\tilde{p}$ is treated as fixed (estimated online via Equation 1), the only dependence on W_{ij} enters through $\tilde{T}_{ij}$, and hence through q_m . We first compute $\frac{\partial q_m}{\partial W_{ij}}$.

Recall $q_m = \sum_{a \in m} \sum_{b \notin m} \tilde{T}_{ab} \tilde{p}_a$. Since W_{ij} appears in $\tilde{T}_{ik} = \tilde{W}_{ik} / \tilde{s}_i$ for all k , we have two cases depending on whether $i \in m$:

$$\frac{\partial \tilde{T}_{ik}}{\partial W_{ij}} = \frac{\langle z_j \rangle \delta_{kj}}{\tilde{s}_i} - \frac{\tilde{W}_{ik} \langle z_j \rangle}{\tilde{s}_i^2} = \frac{\langle z_j \rangle}{\tilde{s}_i} \left(\delta_{kj} - \tilde{T}_{ik} \right)$$

If $i \in m$ and $j \notin m$, then W_{ij} increases the exit probability from m :

$$\frac{\partial q_m}{\partial W_{ij}} = \tilde{p}_i \frac{\langle z_j \rangle}{\tilde{s}_i} \left(1 - \tilde{T}_{ij} \right) - \tilde{p}_i \sum_{k \notin m, k \neq j} \tilde{T}_{ik} \frac{\langle z_j \rangle}{\tilde{s}_i} = \frac{\tilde{p}_i \langle z_j \rangle}{\tilde{s}_i} (1 - q_m^i)$$

where $q_m^i = \sum_{k \notin m} \tilde{T}_{ik}$ is the exit probability from community m for node i . If $i \in m$ and $j \in m$, then W_{ij} redirects flow inward, decreasing q_m :

$$\frac{\partial q_m}{\partial W_{ij}} = - \frac{\tilde{p}_i \langle z_j \rangle}{\tilde{s}_i} q_m^i$$

Now we differentiate each term of $L(M)$. Let $m(i)$ denote the community containing node i . Using $\frac{d}{dx}[x \log x] = 1 + \log x$:

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial}{\partial W_{ij}} \left[-\sum_m q_m \log \frac{q_m}{q_{\sim}} + \sum_m q_m \log \frac{q_m}{p_m + q_m} + \sum_m \sum_{a \in m} \tilde{p}_a \log \frac{\tilde{p}_a}{p_m + q_m} \right]$$

Differentiating the first term, treating $q_{\sim} = \sum_m q_m$:

$$\frac{\partial}{\partial W_{ij}} \left[-\sum_m q_m \log \frac{q_m}{q_{\sim}} \right] = -\frac{\partial q_{m(i)}}{\partial W_{ij}} \log \frac{q_{m(i)}}{q_{\sim}}$$

Differentiating the second term:

$$\frac{\partial}{\partial W_{ij}} \left[\sum_m q_m \log \frac{q_m}{p_m + q_m} \right] = \frac{\partial q_{m(i)}}{\partial W_{ij}} \log \frac{q_{m(i)}}{p_{m(i)} + q_{m(i)}}$$

Differentiating the third term (only $p_m + q_m$ depends on W_{ij} via q_m):

$$\frac{\partial}{\partial W_{ij}} \left[\sum_m \sum_{a \in m} \tilde{p}_a \log \frac{\tilde{p}_a}{p_m + q_m} \right] = -\frac{\partial q_{m(i)}}{\partial W_{ij}} \cdot \frac{\sum_{a \in m(i)} \tilde{p}_a}{p_{m(i)} + q_{m(i)}} = -\frac{\partial q_{m(i)}}{\partial W_{ij}} \cdot \frac{p_{m(i)}}{p_{m(i)} + q_{m(i)}}$$

Combining all three terms:

$$\begin{aligned} \frac{\partial L}{\partial W_{ij}} &= \frac{\partial q_{m(i)}}{\partial W_{ij}} \left[-\log \frac{q_{m(i)}}{q_{\sim}} + \log \frac{q_{m(i)}}{p_{m(i)} + q_{m(i)}} - \frac{p_{m(i)}}{p_{m(i)} + q_{m(i)}} \right] \\ &= \frac{\partial q_{m(i)}}{\partial W_{ij}} \left[\log \frac{q_{\sim}}{p_{m(i)} + q_{m(i)}} - \frac{p_{m(i)}}{p_{m(i)} + q_{m(i)}} \right] \end{aligned}$$

Substituting $\frac{\partial q_{m(i)}}{\partial W_{ij}}$ from above, the gating term $G(i, j)$ is:

$$G(i, j) = \frac{\tilde{p}_i \langle z_j \rangle}{\tilde{s}_i} \cdot \sigma_{ij} \cdot \left[\log \frac{q_{\sim}}{p_{m(i)} + q_{m(i)}} - \frac{p_{m(i)}}{p_{m(i)} + q_{m(i)}} \right]$$

However, in real time we cannot compute $\langle z_j \rangle$, $\tilde{p}_i$, or $q_{m(i)}^i$ directly. We address each in turn, guided by a key distinction: *spike-driven terms* should use instantaneous spike variables $z(t)$, while *flow-structural terms* should use the stationary distribution estimates $\tilde{p}$.

The $\langle z_j \rangle$ factor in $G(i, j)$ is a spike-driven term — it acts as a synaptic eligibility trace, gating the weight update by whether neuron j actually fired at the moment of the update. Replacing it with $z_j(t)$ preserves this temporal locality, which is essential for STDP-like behavior. Similarly, $\tilde{p}_i$ is replaced by its online estimate from Equation 1.

In contrast, $q_{m(i)}^i = \sum_{k \notin m(i)} T_{ik}$ is a flow-structural term — it represents the fraction of node i 's outgoing flow that leaves its community, a time-averaged property

of the network structure. Using instantaneous spikes $z_k(t)$ here would produce a highly noisy and biased estimate, since most neurons are silent at any given moment. Instead, we substitute $\langle z_k \rangle \approx \tilde{p}_k(t)$. Expanding using $\tilde{T}_{ik} = \tilde{W}_{ik}/\tilde{s}_i$:

$$q_{m(i)}^i = \frac{\sum_{k \notin m(i)} W_{ik} \langle z_k \rangle}{\sum_k W_{ik} \langle z_k \rangle}$$

Substituting $\langle z_k \rangle \approx \tilde{p}_k(t)$, which is already maintained online, gives the real-time approximation:

$$q_{m(i)}^i \approx \frac{\sum_{k \notin m(i)} W_{ik} \tilde{p}_k(t)}{\tilde{s}_i} \quad (3)$$

This is computable from quantities already tracked online: the weights W_{ik} , the running stationary distribution estimates $\tilde{p}_k(t)$, the normalization $\tilde{s}_i$, and the community membership of each node. The final form of the gating term is:

$$G(i, j) = \frac{\tilde{p}_i(t) z_j(t)}{\tilde{s}_i} \cdot \sigma_{ij} \cdot \left[\log \frac{q_{\sim}}{p_{m(i)} + q_{m(i)}} - \frac{p_{m(i)}}{p_{m(i)} + q_{m(i)}} \right] \quad (4)$$

where:

$$\sigma_{ij} = \begin{cases} 1 - q_{m(i)}^i & \text{if } j \notin m(i) \text{ (cross-community synapse)} \\ -q_{m(i)}^i & \text{if } j \in m(i) \text{ (within-community synapse)} \end{cases} \quad (5)$$

with $q_{m(i)}^i$ computed via Equation 3.

This gives the full Map-STDP rule:

$$\Delta W_{ij} = \eta \cdot \text{STDP}(i, j) - \eta' \cdot G(i, j) \quad (6)$$

The two learning rates η and η' control the relative contribution of each objective. The gradient term $G(i, j)$ has an intuitive interpretation: it is positive for cross-community synapses when the community is over-represented in the exit flow ($q_{\sim} > p_{m(i)} + q_{m(i)}$), so subtracting it decreases cross-community weights and increases within-community weights, thus encouraging the emergence of modular structure.

4 Training

Now that we have derived the Map-STDP rule, we can train an SNN using this rule. We however have multiple update steps that are interdependent: the synaptic updates depend on the online estimate of $\tilde{p}$, which in turn depends on the community assignments of the nodes. To address this, we use an alternating optimization approach between $\tilde{p}$ and W_{ij} , and every T steps we can reassign communities, and repeat this process until convergence.

Furthermore, although the Map-STDP rule was derived from Infomap's map equation, it can be used with any community detection algorithm, as it only requires a community assignment M regardless of how it was generated. If the application requires a fixed set of outputs (e.g., classification), a clustering algorithm with a fixed number of communities can be used. However, for applications involving unsupervised

representation learning, such as an SNN processing natural visual or auditory stimuli where the number of meaningful concepts is unknown and may change over time, a variable- k method is more appropriate. In such settings, a fixed number of communities would either merge concepts that should be distinct or artificially split those that should not be, whereas a variable- k algorithm like Leiden or Louvain naturally grows the number of communities as the weight structure becomes more differentiated, allowing the modular structure to reflect the true complexity of the input distribution. In either case, the community assignments are updated every T steps as described above.

The starting network can be initialized using a synthetic network generator. The type of network generator can be chosen based on the application. For example, if we require a constant number of communities, we can use the stochastic block model. If we require a variable number of communities, we can use the LFR benchmark. The network can also be initialized with random weights, which will then be updated using the Map-STDP rule.

References

- [1] Shouval, H.Z., Wang, S.S.-H., Wittenberg, G.M.: Spike timing dependent plasticity: a consequence of more fundamental learning rules. *Front. Comput. Neurosci.* **4** (2010)
- [2] Buesing, L., Bill, J., Nessler, B., Maass, W.: Neural dynamics as sampling: a model for stochastic computation in recurrent networks of spiking neurons. *PLoS Comput. Biol.* **7**(11), 1002211 (2011)
- [3] Rosvall, M., Bergstrom, C.T.: Maps of random walks on complex networks reveal community structure. *Proc. Natl. Acad. Sci. U. S. A.* **105**(4), 1118–1123 (2008)
- [4] Dayan, P., Abbott, L.F.: Information theory. In: *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*, pp. 123–150. MIT Press, Cambridge, MA (2001). <https://doi.org/10.7551/mitpress/6755.001.0001>
- [5] Friston, K.: The free-energy principle: a unified brain theory? *Nat. Rev. Neurosci.* **11**(2), 127–138 (2010)

Appendix A Proofs and Remarks

Theorem 1 *The local update rule in Equation 1 converges to the stationary distribution of the random walk on the network.*

Proof Taking the expectation of the second term in the local update rule:

$$\mathbb{E} \left[\sum_j \tilde{T}_{ij} z_j(t) \right] = \sum_j \tilde{T}_{ij} \langle z_j \rangle = \sum_j \tilde{T}_{ij} \tilde{p}_j = [\tilde{T} \tilde{p}]_i$$

Substituting back into the local update rule gives:

$$\tilde{p}_i \leftarrow (1 - \beta)\tilde{p}_i + \beta[\tilde{T}\tilde{p}]_i$$

By the fixed point property of the stationary distribution $\tilde{p} = \tilde{T}\tilde{p}$, this reduces to:

$$\tilde{p}_i \leftarrow (1 - \beta)\tilde{p}_i + \beta\tilde{p}_i = \tilde{p}_i$$

so $\tilde{p}$ is a fixed point of the update rule. $\square$

Theorem 2 *For fixed W , the online estimate $\tilde{p}_k(t)$ is an asymptotically unbiased approximator for $\langle z_k \rangle = p(z_k = 1 \mid \mathbf{o})$, i.e., $\lim_{t \rightarrow \infty} \mathbb{E}[\tilde{p}_k(t)] = \langle z_k \rangle$.*

Proof Unrolling the recursion in Equation 1 gives:

$$\mathbb{E}[\tilde{p}_i(t)] = (1 - \beta)^t \tilde{p}_i(0) + \beta \sum_{s=0}^{t-1} (1 - \beta)^{t-1-s} \mathbb{E} \left[\sum_j \tilde{T}_{ij} z_j(s) \right]$$

Since W is held fixed, $\mathbb{E}[\sum_j \tilde{T}_{ij} z_j(s)] = \sum_j \tilde{T}_{ij} \langle z_j \rangle = \tilde{p}_i$ by Theorem 1. Substituting:

$$\mathbb{E}[\tilde{p}_i(t)] = (1 - \beta)^t \tilde{p}_i(0) + \tilde{p}_i \cdot \beta \sum_{s=0}^{t-1} (1 - \beta)^{t-1-s}$$

Evaluating the geometric sum $\beta \sum_{s=0}^{t-1} (1 - \beta)^{t-1-s} = 1 - (1 - \beta)^t$:

$$\mathbb{E}[\tilde{p}_i(t)] = (1 - \beta)^t \tilde{p}_i(0) + \tilde{p}_i \left(1 - (1 - \beta)^t \right)$$

Since $0 < \beta \leq 1$, we have $(1 - \beta)^t \rightarrow 0$ as $t \rightarrow \infty$, and therefore:

$$\lim_{t \rightarrow \infty} \mathbb{E}[\tilde{p}_i(t)] = \tilde{p}_i = \langle z_i \rangle$$

where the last equality follows from Buesing et al. [2], who establish that the stationary visit frequency of the random walk coincides with the marginal activation probability $p(z_i = 1 \mid \mathbf{o})$. $\square$